Explain the project with evidence

Battery thermal-management simulation, controller comparison and viva preparation

Prepared for Harshit Singh Parmar
Reinforcement Learning-Based Dynamic Battery Thermal Management for Software Defined Electric Vehicles

What problem does this project address?

Driving and fast charging generate battery heat. Cooling must manage temperature limits and differences between modules while consuming as little auxiliary electricity as practical. This project compares learned and conventional cooling controllers under identical simulated conditions.

A 45-second opening explanation

"Our project implements a coupled electrical and thermal battery simulator with four thermal regions and a finite coolant loop. Controllers independently command a pump, fan and chiller every five seconds. We trained PPO, SAC and TD3 across five seeds each, then compared them with PID, bang-bang, no cooling and sampling MPC on eight fixed cases, with and without a separate safety supervisor. The dashboard runs saved controllers through fresh Python simulations and exposes the resulting trajectories and evidence. At the current budget, RL did not outperform tuned PID; supervision accounts for the learned policies' recorded safety."

What the project actually contains

- **Plant:** road-load demand, an electrical equivalent circuit, state of charge, four thermal nodes and serial coolant warming.
- **Control:** three continuous commands; 1-second physics updates and 5-second controller decisions.
- **Study:** 15 trained policies, retained checkpoints and logs, and 304 final evaluation outcomes.
- **Demonstration:** a local dashboard for fresh rollouts, comparisons, playback and JSON/CSV export.

The evidence boundary

Retained EPA driving speed traces are source inputs. Battery temperatures, cooling energy and actions are calculated model outputs. The study does not demonstrate physical battery tests or automotive-ECU certification.

Reading rule: comparison COST lower is better; training RETURN higher is better. Return is negative accumulated cost.

What the simulation shows

A fresh calculation of the battery response under one selected episode and controller.

Dashboard item	How to explain it
Four module temperatures	Four modeled thermal regions, not four physical temperature sensors. They show both peak temperature and uneven heating.
Coolant temperature	The finite coolant inventory warms as it absorbs battery heat and cools through heat rejection.
State of charge (SoC)	Remaining modeled battery charge, shown as a percentage.
Pump / fan / chiller	Independent normalized actuator commands from 0% to 100%. These are commands, not measured hardware speeds.
Policy request / applied command	What the controller asks for versus what is applied after possible safety supervision.
Cooling energy / violations	Accumulated cooling electricity in Wh and seconds outside the 15-45 C band.
Temperature chart	Four lines for the primary controller. The comparison line is the other controller's maximum module temperature.
Recorded-result match	Whether the fresh calculation agrees with the corresponding saved metrics within the documented numeric tolerance.

What happens after Run fresh simulation?

Python loads the selected fixed episode and existing controller. For RL it loads the saved neural policy for the selected training seed. It calculates the rollout and compares it with the recorded result. It does not train a policy or rewrite the study table.

What does playback do?

Playback animates the calculated trajectory at an accelerated rate. It does not create new measurements. JSON export contains the run evidence and comparisons; CSV export contains trajectory samples and actuator requests/applied commands.

The local Python backend supports fresh simulation. A static evidence snapshot supports recorded results/replays only. Source: btms/dashboard/simulation.py and btms/dashboard/web/.

The seven controllers

Three learned methods and four conventional or reference controllers.

Controller	Meaning and behavior in this implementation
Sampling MPC	Model Predictive Control. Predicts future outcomes for sampled cooling commands and chooses the lowest-cost candidate. No RL training.
Tuned PID	Proportional-Integral-Derivative feedback. Tuning selected a 39 C set-point, Kp=0.6 and Ki=Kd=0, so this chosen setting uses proportional feedback. Actuator routing considers ambient/coolant conditions.
TD3	Twin Delayed Deep Deterministic Policy Gradient. Learns continuous commands using two value estimators, delayed policy updates and target-policy smoothing.
SAC	Soft Actor-Critic. Learns continuous commands by balancing expected reward and policy entropy during training.
PPO	Proximal Policy Optimization. Learns a policy using updates that limit abrupt changes, through clipping.
No cooling	The null controller requests zero pump, fan and chiller command. Supervision can still force cooling if enabled.
Bang-bang	Thermostat-style hysteresis: all cooling on at maximum module temperature >=39 C; stays on until maximum temperature <=35 C.

Sampling MPC: exactly what it samples

At each decision it tests 10 command combinations: zero cooling, full cooling, its previous command and seven randomly sampled combinations. Each candidate is held constant in a copied simulator for up to 12 decisions, or 60 simulated seconds. It selects the lowest predicted cost, applies that command for the next interval and repeats.

This is a limited sampling comparator, not a certified global optimum. It uses the nominal simulator state and future demand, which gives it an information advantage over a policy observing the defined observation vector.

What the learned policies observe and output

The 12-value observation includes four temperatures, SoC, C-rate, ambient temperature, charging flag, coolant temperature, current power demand and two future-demand summaries. The neural policy outputs three bounded actuator commands. Saved evaluations use deterministic policy predictions.

Sources: btms/controllers/; btms/envs/btms_env.py; Stable-Baselines3 official PPO, SAC and TD3 documentation (links on page 8).

Safety and metric meanings

Supervisor enabled versus disabled

Enabled: when the hottest module is >=42 C, a separate heuristic rule applies 100% pump, fan and chiller. Below the threshold, the controller's requested action is applied. **Disabled:** only the controller's own commands are applied. The safety band used for violation measurement is 15-45 C.

"Supervised" here refers to safety intervention. It does not mean supervised machine learning or training from labeled examples.

Metric	Meaning	Better direction
MEAN COST	Average accumulated normalized objective: temperature-limit penalties, spatial temperature spread, cooling electricity and relative aging exposure. Not money or Wh.	Lower
AUX. Wh	Average electrical energy consumed by the cooling equipment. Not total driving energy.	Lower, while safe
VIOLATION %	Time with any module below 15 C or above 45 C, divided by episode duration, times 100.	Lower; ideally 0%
OVERRIDE %	Time that the safety rule is active, divided by episode duration, times 100. It can activate even if the raw command already equals full cooling.	Generally lower, while safe
SEEDS x EPISODES	How many training-seed variants and fixed episode cases contributed to that row.	Evidence count; no performance direction

How the score is constructed

Each 1-second update adds a weighted cost: 20 times the normalized squared temperature-band exceedance, plus normalized module spread, cooling power and relative aging exposure. Reward is the negative increment. Lower total cost therefore gives a higher, less negative return.

Interpret the numbers together

- Zero violations with supervision does not establish that the learned controller is independently safe.
- Zero override when supervision is disabled is automatic; it is not a safety achievement.
- Low energy caused by inadequate cooling is not good performance.
- A full-cooling rule cannot guarantee safety under every possible thermal load.

Percentages are equal-episode-weight averages, not pooled time-weighted rates. For RL, episode means are averaged across the five training seeds. Source: btms/dashboard/evidence.py.

What the saved results say

Original final-evaluation means, rounded to two decimals. Lower mean cost is better.

Supervisor enabled

Controller	Mean cost	Aux. Wh	Violation %	Override %
Sampling MPC	1,618.36	445.26	0.00	14.31
Tuned PID	1,737.38	482.31	0.00	1.23
TD3	1,750.04	483.99	0.00	21.85
SAC	1,768.50	490.39	0.00	19.63
No cooling	1,776.07	473.85	0.00	22.68
PPO	1,841.61	474.76	0.00	22.70
Bang-bang	1,983.07	626.79	0.00	1.23

Supervisor disabled

Controller	Mean cost	Aux. Wh	Violation %	Override %
Tuned PID	1,735.41	480.71	0.00	0.00
Bang-bang	1,983.07	626.79	0.00	0.00
Sampling MPC	2,897.62	308.57	42.15	0.00
SAC	54,951.44	97.10	41.15	0.00
TD3	105,270.10	38.36	42.08	0.00
PPO	140,570.28	0.35	42.27	0.00
No cooling	144,062.77	0.00	42.28	0.00

The interpretation to present

- **Sampling MPC** has the lowest supervised mean cost, with substantially more computation and model/future-demand access.
- **Tuned PID** has the lowest unsupervised mean cost and zero recorded violations on these eight cases.
- **TD3** has the lowest supervised mean cost among the three learned methods.
- All three learned methods have higher supervised mean cost than tuned PID. Without supervision they violate the band for more than 41% of time on average.

The supported conclusion is a comparative simulation study with measured supervisory dependence. These results do not demonstrate an RL advantage over tuned PID at the saved training budget.

Source: report/results/controller_comparison.csv, checked against the individual retained evaluation records. Comparisons apply to the tested model, cases, tuning and training budget.

Runs, policies and episodes

Training, checkpoint storage and final evaluation are different counts.

Term	Meaning
Algorithm / run	The learning method / one execution of training with one algorithm and seed.
Training seed / policy	A setting controlling training randomness / the learned controller produced by a run. Seeds are 0, 1, 2, 3 and 4 per RL algorithm.
Episode / checkpoint	A complete driving or charging rollout / a saved model snapshot. A new snapshot is not an independent training run.

Method	Training runs / final policies	Steps per run	Results per mode	Both modes
PPO	5 / 5	20,480	5 x 8 = 40	80
SAC	5 / 5	20,000	5 x 8 = 40	80
TD3	5 / 5	20,000	5 x 8 = 40	80
Tuned PID	0 / no learned policy	-	8	16
Sampling MPC	0 / no learned policy	-	8	16
Bang-bang	0 / no learned policy	-	8	16
No cooling	0 / no learned policy	-	8	16
TOTAL	15 / 15	302,400 total	152	304

The important counting details

- PPO finished its rollout batch at 20,480 steps rather than the requested 20,000. Steps are environment transitions, not episodes or gradient updates.
- Each run used 8 parallel simulator environments. These workers do not produce 8 separate learned policies; recorded steps already count across the workers.
- There are 30 model.zip snapshots: 15 intermediate and 15 final. Resume pointers and exported formats do not create additional independent policies.
- A baseline's displayed 1 x 8 means one configuration on eight cases, not one RL training seed.

Completed training episodes: what can be verified

SAC and TD3 final checkpoints each record an episode counter of 63 per run, or 315 per algorithm. PPO's stored counter is not usable for this count; its zero does not mean no episodes occurred. A verified completed-training-episode total across all three algorithms is therefore unavailable.

Test cases and evidence

Eight fixed evaluation cases

Conditions	Cases	Episode identifiers
Mild UDDS driving	2	mild_s701, mild_s702
Hot US06 driving	2	hot_s701, hot_s702
Fast charging	2	fastcharge_s701, fastcharge_s702
Held-out NEDC driving	1	heldout_nedc
HWFET driving	1	evaluation_hwfet

The paired cases differ in fixed initial temperature and SoC. Every policy sees the same manifest. Forty outcomes for one RL method in one mode means five policies tested on eight cases, not forty different scenarios. Episode seeds 701-704 are separate from training seeds 0-4.

Training curves and tuning

The 15 curves contain 50 distinct recorded tuning-evaluation return points. A point summarizes evaluation on the separate tuning split; it is not one training episode. Connecting lines are a plotting aid, not extra observations. Higher return is better. NEDC is held out from training; UDDS, HWFET and US06 are training cycles.

What was verified previously

Record	What it establishes
15 Python tests passed	Recorded tests cover plant/normalizer behavior and dashboard/evidence boundaries. This guide creation did not retrain the models or rerun that test suite.
288 matching reproduced outcomes	Fresh replay of all saved non-MPC final evaluations: 240 RL outcomes plus 48 PID/null/bang-bang outcomes. Numeric tolerance was used; duration, violation and supervisor seconds had to match exactly.
MPC reproduction spot-check	One separate HWFET replay matched. The full 16-outcome MPC evaluation suite was not freshly rerun in that reproduction audit.
302,400 recorded training steps	Saved run metadata, checkpoints and training scalar events support the experiment counts. Additional dashboard runs do not increase this training total.

Limits to state openly

Thermal/cooling parameters include engineering assumptions. Aging is a dimensionless relative exposure proxy, not measured capacity loss. Laptop inference timings do not certify an automotive ECU. Public evaluator-facing repository/data/demo links and the submission venue remain undecided.

Repeated evaluations and reproduction runs are checks, not additional independent trained policies or new scenario samples.

Present it confidently

A practical five-minute demonstration

- **1. Explain the objective:** keep module temperatures within the study band while managing uneven heating and cooling electricity.
- **2. Select hot_s701 and tuned PID, supervisor disabled:** run it and explain temperature feedback, the 39 C set-point and the cost/violation metrics.
- **3. Compare SAC with PID, supervisor disabled:** show the freshly calculated difference on the same case; explain the saved study's raw-policy limitation.
- **4. Repeat with supervision enabled:** point out the 42 C trigger and distinguish requested commands from applied full cooling.
- **5. Open study results and training evidence:** explain 5 seeds x 8 cases, show a checkpoint/configuration and export a run JSON or trace CSV.

Likely viva questions: short answers

What is the contribution? The implemented coupled simulation, comparison protocol, saved multi-seed experiments, safety-intervention analysis and inspectable demonstration. These use existing RL algorithms; do not claim a new RL algorithm. Describe your personal work separately from team work.

Why can PID beat RL? PID was tuned for this objective and model. The recorded short-budget RL policies underperform it. Longer training or design changes need new experiments; improvement is not guaranteed.

Why is this still useful? It establishes a reproducible comparison and exposes dependence on a safety supervisor. An honest negative result is a research finding; its value depends on the method and evidence.

What would you improve? Run longer-training and reward/controller-design experiments, then repeat matched safety and robustness tests. Validate uncertain model parameters with independent data or hardware when available.

Where to point an evaluator

Local project root: D:/Coding/RL BATTERY SYSTEM/. Paths below are evidence locations, not public-access links. Evaluator sharing still needs to be arranged.

Simulation: btms/dashboard/simulation.py; btms/envs/btms_env.py
Controllers: btms/controllers/; btms/envs/shield.py
Results: report/results/controller_comparison.csv; policy_evaluations.json
Cases: data/evaluation_episodes.json
Runs/checkpoints/configs/events: outputs/study_20000/
Tests/reproduction: report/dashboard/test_report.json; reproduction.json; mpc_spotcheck.json

Official algorithm references: PPO documentation; SAC documentation; TD3 documentation. Project-specific behavior and numbers come from the local code and saved artifacts.